

JavaScript

Conoce este lenguaje de
programación

Astrid Martinez
Linda Vidal

A large yellow square is positioned on the right side of the slide. Inside this square, the letters 'JS' are written in a large, bold, black sans-serif font, representing the JavaScript logo.

ÍNDICE

- 01.** Declaración de variables (var, let,
- 02.** const)
- 03.** Tipos primitivos (string, number,
- 04.** boolean)
- 05.** Operadores aritméticos, de
- 06.** comparación y lógicos
- 07.** Precedencia de operadores
- 08.** Condicionales (if-else, switch)
- 09.** Bucles (for, while, do-while)
- 10.** Declaración y expresión de
- 11.** funciones
- 12.** Parámetros y retorno de valores
- 13.** Creación y manipulación de Arrays
- 14.** Métodos principales (push, pop,
- 15.** map, filter)
- 16.** Creación de objetos
- 17.** Acceso a propiedades y métodos
- 18.** Selección y manipulación de
- 19-** elementos
- 20.** Manejo de eventos básicos
- 21.** Callbacks
Promesas básicas

{.js}

JavaScript

DECLARACIÓN DE VARIABLES EN JAVASCRIPT (VAR, LET, CONST)

var: Antiguo. Tiene alcance de función, no de bloque. Puede redeclararse y cambiar de valor.

let: Tiene alcance de bloque. Puede cambiar su valor, pero no puede redeclararse dentro del mismo bloque.

const: También tiene alcance de bloque, pero no se puede cambiar su valor después de ser declarado. Es ideal para valores constantes.

TIPOS PRIMITIVOS EN JAVASCRIPT

string (cadena de texto): Texto entre comillas. Ej: "Hola", 'JavaScript'

number (número): Números enteros o decimales. Ej: 10, 3.14

boolean (booleano): Representa verdadero o falso Ej: true, false

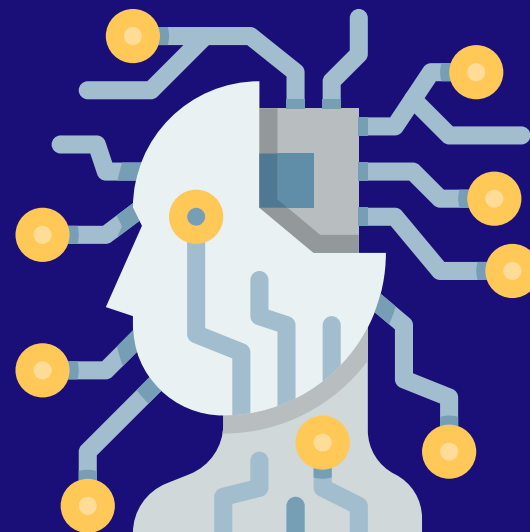
EJEMPLOS DE DECLARACION DE VARIABLES Y TIPOS PRIMITIVOS

DECLARACION

```
var nombre = "Ana";  
let edad = 25;  
const PI = 3.14;
```

TIPOS

```
let nombre = "Luis";    // string  
let edad = 30;          // number  
let esEstudiante = true; // boolean
```



OPERADORES ARITMETICOS, COMPARACION Y LOGICOS.

ARITMETICOS

Se usan para realizar operaciones matemáticas.

- + Suma
- - Resta
- * Multiplicación
- / División
- % Módulo (resto de una división)
- ** Exponente

COMPARACIÓN

Comparan dos valores y devuelven un booleano (true o false).

- == Igualdad (sin verificar tipo)
- === Igualdad estricta (verifica tipo y valor)
- != Diferente
- !== Diferente estricto
- > Mayor que
- < Menor que
- >= Mayor o igual
- <= Menor o igual

LOGICOS

Se usan para combinar condiciones.

- && (AND): Verdadero si ambas condiciones son verdaderas
- || (OR): Verdadero si al menos una condición es verdadera
- ! (NOT): Invierte el valor lógico

EJEMPLOS

ARITMETICOS

📌 Ejemplo:

```
let resultado = 5 + 3; // 8
```

COMPARACIÓN

📌 Ejemplo:

```
5 === "5"; // false  
5 == "5"; // true
```

LOGICOS

📌 Ejemplo:

```
true && false; // false  
true || false; // true  
!true;        // false
```

CONDICIONALES: IF - ELSE

Permite ejecutar un bloque de código si se cumple una condición.

Ejemplo

```
let edad = 18;  
if (edad >= 18) {  
  console.log("Eres mayor de edad");  
} else {  
  console.log("Eres menor de edad");  
}
```

CONDICIONALES: SWITCH

Se usa para comparar un valor con varios posibles casos.

 **Ejemplo:**

```
let dia = "lunes";
```

```
switch (dia) {  
  case "lunes":  
    console.log("Inicio de semana");  
    break;  
  case "viernes":  
    console.log("Fin de semana cercano");  
    break;  
  default:  
    console.log("Otro día");  
}
```

BUCLAS (LOOPS)

FOR

Se usa cuando sabes cuántas veces quieres repetir algo.

📌 Ejemplo:

```
for (let i = 0; i < 5; i++) {  
  console.log("Repetición " + i);  
}
```

WHILE

Repite mientras una condición sea verdadera.

📌 Ejemplo:

```
let i = 0;  
while (i < 3) {  
  console.log("Valor: " + i);  
  i++;  
}
```

DO-WHILE

Ejecuta el bloque al menos una vez, y luego repite mientras la condición sea verdadera.

📌 Ejemplo:

```
let i = 0;  
do {  
  console.log("Número: " + i);  
  i++;  
} while (i < 2);
```

DECLARACIÓN Y EXPRESIÓN DE FUNCIONES

Se utiliza la palabra clave `function` seguida del nombre de la función. Estas funciones se elevan (hoisting), lo que significa que pueden ser llamadas antes de su declaración en el código.

Las declaraciones de funciones se utilizan típicamente para funciones que se ejecutarán en múltiples lugares, mientras que las expresiones de función son más comunes para funciones que se usan como argumentos o dentro de otras expresiones.

EJEMPLO DE DECLARACIÓN

```
function calcularCuadrado(numero) {  
    return numero * numero;  
}
```

```
console.log(calcularCuadrado(4)); // Salida: 16
```

Usamos la palabra clave **function** y le damos un nombre: **calcularCuadrado**.

La función recibe un argumento (**numero**).
Como es una declaración, podemos llamarla antes o después de haberla definido

EJEMPLO DE EXPRESIÓN

```
const calcularCuadrado = function(numero) {  
    return numero * numero;  
};
```

```
console.log(calcularCuadrado(4)); // Salida: 16
```

En lugar de declarar la función directamente, la almacenamos en una variable `calcularCuadrado`. No tiene un nombre explícito, es anónima y se asigna a `const`.

Solo podemos llamarla después de haberla definido (no se eleva como la declaración).

PARÁMETROS Y RETORNO DE VALORES

Los Parámetros son variables que una función recibe al ser invocada. Se definen en los paréntesis al declarar la función y permiten que esta trabaje con diferentes valores.

 **Ejemplo:**

```
function saludar(nombre) {  
  console.log("Hola, " + nombre + "!");  
}
```

```
saludar("Lucia"); // Hola, Lucia!
```

En este caso, **nombre** es un parámetro que recibe un valor al llamar la función.

RETORNO DE VALORES

El Retorno de valores es el resultado que una función envía de vuelta cuando finaliza su ejecución. Se usa la palabra clave return. 📌 **Ejemplo:**

```
function multiplicar(a, b) {  
    return a * b;  
}  
  
let resultado = multiplicar(5, 3);  
console.log(resultado); // 15
```

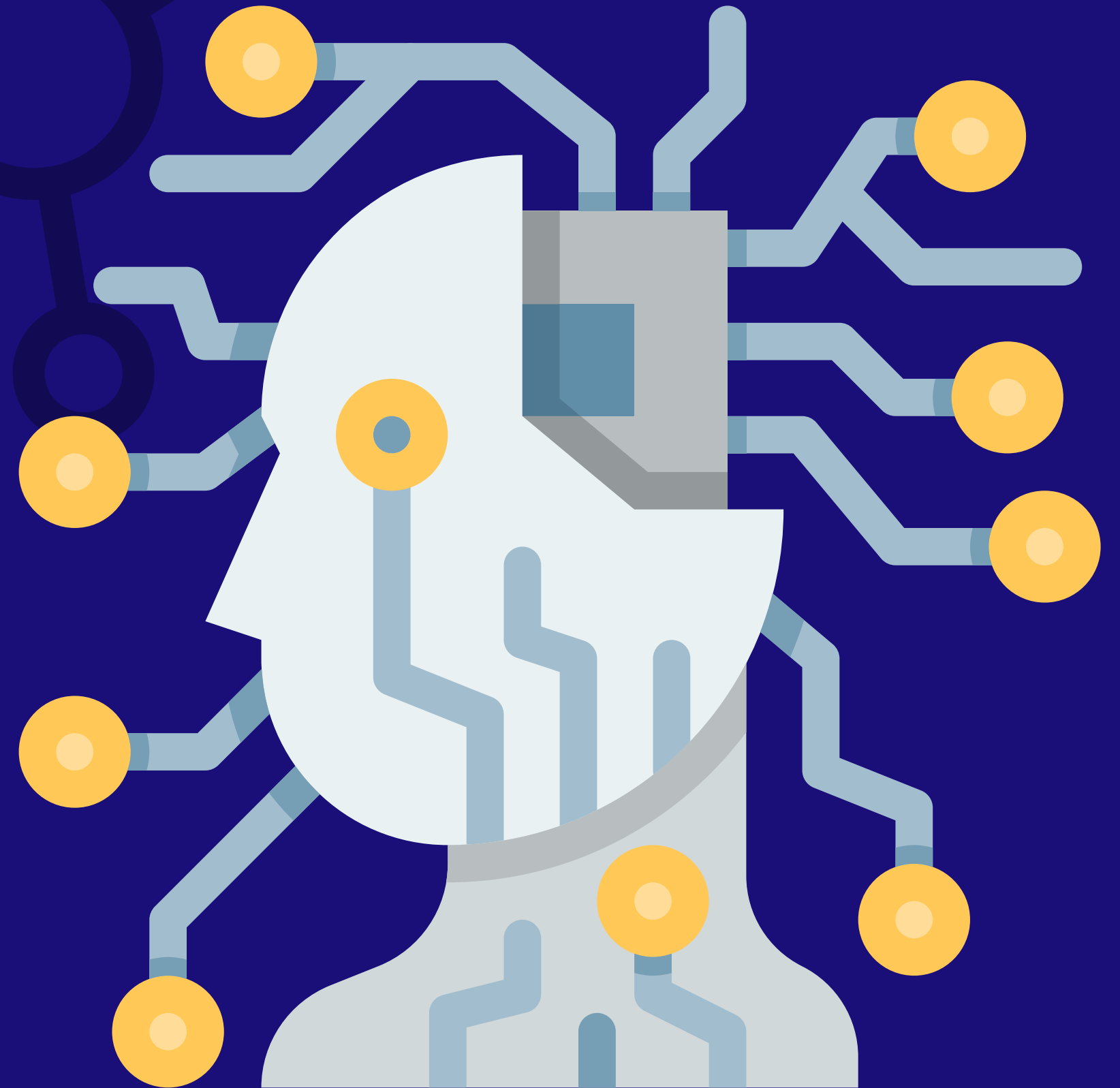
Aquí, la función **multiplicar** devuelve la multiplicación de **a** y **b**, almacenando el resultado en la variable **resultado**.

ARRAYS

Un array (o arreglo) es una estructura que sirve para guardar varios datos en una sola variable, en lugar de tener muchas variables separadas.

📌 Ejemplo:

```
let frutas = ("manzana", "banana",  
"pera");  
donde:  
frutas(0) es "manzana"  
frutas(1) es "banana"  
frutas(2) es "pera"
```



CREACIÓN Y MANIPULACIÓN DE ARRAYS

CREACIÓN

Un array se crea usando corchetes [], y dentro se colocan los elementos separados por comas.

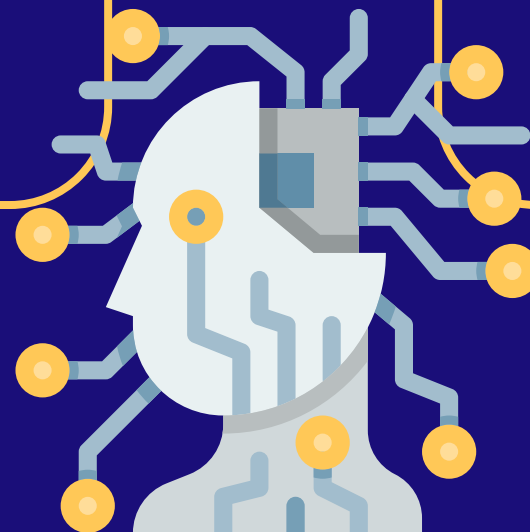
```
let numeros = (10, 20, 30, 40);
```

Cada elemento tiene una posición (llamada índice) que empieza desde 0:

```
let colores = ("rojo", "verde", "azul");  
console.log(colores(0)); // "rojo"  
console.log(colores(1)); // "verde"  
console.log(colores(2)); // "azul"
```

MANIPULACIÓN

Significa trabajar con los elementos dentro de un arreglo: podemos agregar elementos usando `push()` o `unshift()`, eliminar con `pop()` o `shift()`, cambiar valores por su índice y recorrerlos con bucles como `for` o métodos como `forEach()`. Además, se pueden crear nuevos arrays aplicando funciones con `map()` o filtrando datos con `filter()`



EJEMPLO

```
let edades = (15, 22, 30, 17, 40);
```

```
edades.push(25);
```

```
edades.shift();
```

```
edades(1) = 35;
```

```
edades.forEach(function(edad) {  
  console.log("Edad:", edad);  
});
```

```
let duplicadas = edades.map(e => e * 2);  
console.log("Edades duplicadas:", duplicadas);
```

```
let mayores = edades.filter(e => e > 30);  
console.log("Edades mayores de 30:", mayores);
```

MÉTODOS PRINCIPALES (PUSH, POP, MAP, FILTER)

push()

Añade uno o más elementos al final de un array.

Ejemplo:

```
let frutas = ("manzana",  
             "pera");  
frutas.push("naranja");  
console.log(frutas); //  
("manzana", "pera",  
 "naranja")
```

pop()

Elimina el último elemento de un array y lo devuelve.

Ejemplo:

```
let colores = ("rojo", "azul",  
              "verde");  
let ultimo = colores.pop();  
console.log(ultimo); //  
"verde"  
console.log(colores); //  
("rojo", "azul")
```

MÉTODOS PRINCIPALES (PUSH, POP, MAP, FILTER.)

map()

Crea un nuevo array aplicando una función a cada elemento del array original.

Ejemplo:

```
let numeros = (1, 2, 3);  
let dobles =  
numeros.map(num => num  
  * 2);  
console.log(dobles); // (2,  
  4, 6)
```

filter()

Crea un nuevo array con los elementos que cumplen una condición.

Ejemplo:

```
let edades = (12, 25, 18, 32,  
  15);  
let mayores =  
edades.filter(edad => edad  
  >= 18);  
console.log(mayores); // (25,  
  18, 32)
```

OBJETOS Y JSON

OBJETOS

Los objetos son estructuras de datos que permiten almacenar múltiples valores en una sola entidad. Se representan como pares clave-valor y pueden contener funciones (métodos) y otros objetos dentro.

◆ Ejemplo de un objeto:

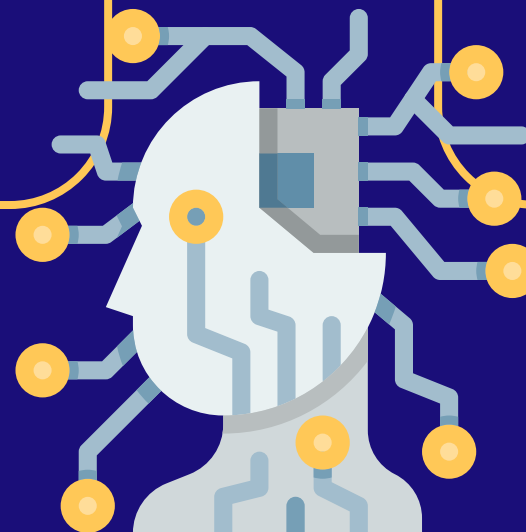
```
let coche = {  
  marca: "Toyota",  
  modelo: "Corolla"  
};  
console.log(coche.marca); // "Toyota"  
console.log(coche.modelo); // "Corolla"
```

JSON (JAVASCRIPT OBJECT NOTATION)

JSON es un formato ligero de intercambio de datos basado en texto que sigue una estructura similar a los objetos en JavaScript, pero con reglas más estrictas.

◆ Ejemplo de JSON:

```
{  
  "nombre": "Ana",  
  "edad": 25,  
  "ciudad": "Córdoba"  
}
```



CREACION DE OBJETOS

Un objeto en JavaScript es una estructura que agrupa datos mediante pares clave-valor. Es una forma de almacenar y organizar información relacionada.

La creación de objetos en JavaScript se puede hacer de diferentes maneras. La forma más común es mediante la notación de llaves {}.

EJEMPLO DE CREACIÓN DE UN OBJETO

```
let persona = {  
  nombre: "Lucia",  
  edad: 25,  
  ciudad: "Córdoba"  
};  
console.log(persona.nombre); // "Lucia"  
console.log(persona.edad); // 25
```

También se pueden crear objetos usando el constructor `new Object()`, aunque es menos común:

```
let coche = new Object();  
coche.marca = "Toyota";  
coche.modelo = "Corolla";  
  
console.log(coche.marca); // "Toyota"
```

ACCESO A PROPIEDADES

Se pueden acceder a las propiedades de un objeto utilizando:

Notación de punto (.)	Notación de corchetes ([])
-----------------------	----------------------------

Ejemplo:

```
let persona = {  
  nombre: "Lucia",  
  edad: 25  
};
```

```
console.log(persona.nombre); // "Ana"  
console.log(persona["edad"]); // 25
```

Ambas formas (`persona.nombre` y `persona["edad"]`) son válidas para obtener valores.

ACCESO A MÉTODOS

Los métodos son funciones dentro de un objeto. Para acceder a ellos, se usa la notación de punto y se ejecuta con `()`.

Ejemplo:

```
let coche = {  
  marca: "Toyota",  
  modelo: "Corolla",  
  mostrarInfo: function() {  
    return "Marca: " + this.marca + ", Modelo: " + this.modelo;  
  }  
};  
  
console.log(coche.mostrarInfo()); // "Marca: Toyota, Modelo:  
Corolla"
```

Aquí, **mostrarInfo()** es un método que usa `this` para acceder a las propiedades del objeto.

DOM Y EVENTOS

DOM

El DOM (Document Object Model) es una representación estructurada de un documento HTML en forma de árbol. Permite a JavaScript acceder y manipular los elementos de una página web de manera dinámica.

Ejemplo de acceso al DOM:

```
document.getElementById("titulo").innerHTML =  
"¡Hola, Lucia!";
```

Aquí, JavaScript selecciona un elemento con el id="titulo" y cambia su contenido.

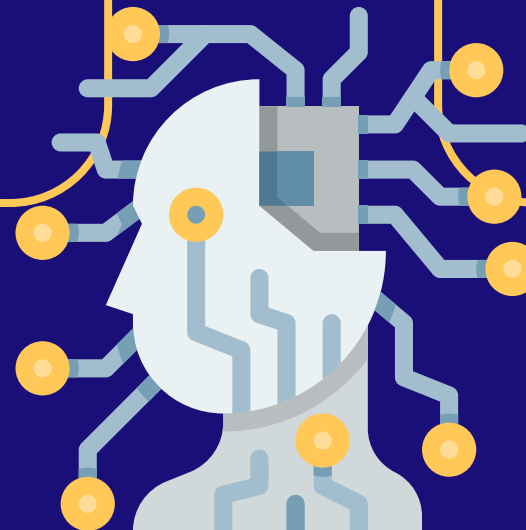
EVENTOS

Los eventos son acciones que ocurren en una página web, como hacer clic en un botón, mover el mouse o presionar una tecla. JavaScript permite reaccionar a estos eventos mediante manejadores de eventos.

Ejemplo de evento:

```
document.getElementById("boton").addEventListener(  
"click", function() {  
    alert("¡Botón presionado!");  
});
```

Cuando el usuario hace clic en el botón con id="boton", se muestra una alerta.



SELECCIÓN DE ELEMENTOS

Para seleccionar elementos del DOM, JavaScript ofrece varios métodos:

getElementById() → Selecciona un elemento por su ID.

```
let titulo = document.getElementById("miTitulo");  
console.log(titulo.innerHTML); // Muestra el contenido del elemento con  
ID "miTitulo"
```

getElementsByClassName() → Selecciona todos los elementos con una clase específica.

```
let elementos = document.getElementsByClassName("miClase");  
console.log(elementos.length); // Muestra cuántos elementos tienen la  
clase "miClase"
```

querySelector() → Selecciona el primer elemento que coincide con un selector CSS.

```
let primerElemento = document.querySelector(".miClase");  
console.log(primerElemento.innerHTML); // Muestra el contenido del  
primer elemento con la clase "miClase"
```

querySelectorAll() → Selecciona todos los elementos que coinciden con un selector CSS.

```
let todosLosElementos = document.querySelectorAll("p");  
console.log(todosLosElementos.length); // Muestra cuántos elementos  
`<p>` hay en la página
```

MANIPULACIÓN DE ELEMENTOS

Una vez seleccionados, podemos modificar su contenido, atributos y estilos.

Modificar contenido con `innerHTML`:

```
document.getElementById("miTitulo").innerHTML = "¡Nuevo título!";
```

Cambiar estilos con `style`:

```
document.getElementById("miTitulo").style.color = "red";
```

Agregar o eliminar clases con `classList`:

```
document.getElementById("miTitulo").classList.add("resaltado");  
document.getElementById("miTitulo").classList.remove("resaltado")  
;
```

Crear y agregar elementos con `createElement()` y `appendChild()`:

```
let nuevoParrafo = document.createElement("p");  
nuevoParrafo.innerHTML = "Este es un nuevo párrafo.";  
document.body.appendChild(nuevoParrafo);
```

MANEJO DE EVENTOS BÁSICOS

El manejo de eventos en JavaScript permite ejecutar acciones cuando ocurren eventos en una página web, como hacer clic en un botón, mover el mouse o presionar una tecla.

EVENTO:

Un evento es cualquier interacción del usuario con la página web. JavaScript permite capturar estos eventos y definir respuestas mediante manejadores de eventos.

Ejemplo de evento:

```
document.getElementById("boton").addEventListener("click", function() {  
    alert("¡Botón presionado!");  
});
```

Explicación:

- `getElementById("boton")` selecciona el botón con `id="boton"`.
- `addEventListener("click", function() {...})` asigna una función que se ejecutará cuando el usuario haga clic en el botón.
- `alert("¡Botón presionado!");` muestra una alerta cuando el botón es presionado.

MANEJO DE EVENTOS BÁSICOS

La asincronía en JavaScript permite ejecutar tareas sin bloquear la ejecución del código, lo que mejora la eficiencia y la experiencia del usuario en aplicaciones web

La programación asíncrona permite que ciertas operaciones, como solicitudes a servidores o manipulación del DOM, se ejecuten en segundo plano mientras el resto del código sigue funcionando

Ejemplo básico de asincronía con setTimeout():

```
console.log("Inicio");

setTimeout(() => {
  console.log("Mensaje después de 2 segundos");
}, 2000);

console.log("Fin");
```

Explicación:

Se imprime "Inicio".	Se programa un mensaje para aparecer
después de 2 segundos.	Mientras tanto, el código sigue
ejecutándose y se imprime "Fin".	Finalmente, después de 2 segundos,
aparece "Mensaje después de 2 segundos".	

CALLBACKS

Una callback es una función que se pasa como parámetro a otra función y se ejecuta dentro de esta última cuando sea necesario.

Las callbacks en JavaScript son funciones que se pasan como argumento a otra función y se ejecutan dentro de ella. Son fundamentales para la programación asíncrona, ya que permiten ejecutar código después de que una operación haya finalizado.

EJEMPLO

```
function procesar(nombre, callback) {  
  console.log("Procesando datos de " + nombre);  
  callback();  
}
```

```
function mostrarMensaje() {  
  console.log("¡Proceso completado!");  
}
```

```
procesar("Lucia", mostrarMensaje);
```

Explicación:

`procesar(nombre, callback)` recibe un nombre y una función como parámetros. Se ejecuta `console.log("Procesando datos de " + nombre)`. Luego, se llama a `callback()`, que ejecuta `mostrarMensaje()`. Se imprime "¡Proceso completado!" después de "Procesando datos de Lucia".

PROMESAS

Una promesa es un objeto que representa el resultado de una operación asíncrona. Puede estar en tres estados: Pendiente (pending) → La operación aún no ha terminado. Resuelta (fulfilled) → La operación se completó exitosamente. Rechazada (rejected) → Hubo un error en la operación.

Las promesas en JavaScript son una forma de manejar operaciones asíncronas, permitiendo ejecutar código cuando una tarea se completa o falla sin bloquear la ejecución del programa.

EJEMPLO

```
let miPromesa = new Promise((resolve, reject) => {  
  let exito = true; // Simula si la operación fue exitosa  
                    // o no  
  
  setTimeout(() => {  
    if (exito) {  
      resolve("¡Operación exitosa!");  
    } else {  
      reject("Hubo un error.");  
    }  
  }, 2000);  
});  
  
miPromesa  
  .then(resultado => console.log(resultado)) // Se  
    ejecuta si la promesa se resuelve  
  .catch(error => console.log(error)); // Se ejecuta si  
    la promesa es rechazada
```

Explicación:

- Se crea una promesa con `new Promise()`, que recibe dos funciones: `resolve` (para éxito) y `reject` (para error).
- Se simula una operación con `setTimeout()`, que después de 2 segundos decide si la operación fue exitosa o fallida.
- Se usa `.then()` para manejar el éxito y `.catch()` para manejar el error.